Multi-Document Comparison RAG

A step-by-step build guide + full source code

This project builds a Retrieval-Augmented Generation (RAG) app that doesn't just answer questions from one document — it retrieves relevant passages from **several** documents independently, then asks an LLM to compare and contrast them. A typical use case: "How do these three research papers differ in their approach to X?"

What makes this different from basic RAG

- Every chunk is tagged with which source document it came from.
- Retrieval runs **per document**, so no single document can dominate the results just because it's larger or more textually similar.
- The prompt sent to the LLM keeps chunks grouped and labeled by source, and explicitly asks the model to compare and cite.

Tech stack

- **Chunking:** simple fixed-size sliding window (pure Python)
- **Embeddings:** sentence-transformers, all-MiniLM-L6-v2 (local, free, no API key)
- **Vector store:** ChromaDB (persistent, local)
- **Generation:** Claude via the Anthropic API
- **Interface:** command-line, using rich for formatted output

Every file referenced in this guide is included in full in the accompanying code folder. Each page below explains one file: what it does, why it's designed that way, and the complete code.

How It Works (Architecture)

The system has two phases: an offline **indexing** phase (run once, or whenever you add documents) and an online **query** phase (run every time a user asks a question).

Indexing phase

1. Load each PDF/text file from data/ → 2. Split into overlapping chunks → 3. Tag each chunk with its source filename → 4. Embed each chunk → 5. Store in ChromaDB with the source tag as metadata.

Query phase

1. User asks a question, e.g. "How do these papers differ in their evaluation methodology?" → 2. For EACH source document, run a separate similarity search filtered to that document, returning its own top-k chunks → 3. Build a prompt where chunks are grouped under clear "SOURCE: filename" headers → 4. Send to Claude with instructions to compare and cite sources → 5. Print the answer.

Why retrieve per-document instead of one big top-k search?

If you embed all documents into one pool and just take the overall top-k most similar chunks, a single long or topically-dominant document can crowd out the others entirely — you might get 8 chunks from Paper A and 0 from Paper B, making comparison impossible. Querying each document's chunks separately guarantees every source gets a fair chance to be represented.

Project structure

```
multidoc_rag/
■■■ data/                # put your PDFs/.txt files to compare here
■■■ chroma_db/           # created automatically -- the persistent vector index
■■■ requirements.txt
■■■ src/
    ■■■ config.py        # all tunable settings in one place
    ■■■ ingest.py        # load documents + chunk them, tagged by source
    ■■■ vector_store.py  # ChromaDB wrapper with per-document querying
    ■■■ compare.py       # builds the comparison prompt + calls Claude
    ■■■ app.py           # CLI entry point that ties it all together
```

Setup: requirements.txt

These are the only dependencies needed. Embeddings run locally via sentence-transformers, so the only API key you need is for Claude (generation).

```
chromadb==0.5.5
sentence-transformers==3.0.1
anthropic==0.34.2
pypdf==4.3.1
tiktoken==0.7.0
python-dotenv==1.0.1
rich==13.7.1
```

Install & run

```
pip install -r requirements.txt
export ANTHROPIC_API_KEY="your-key-here"

# put 2+ PDF or .txt files you want to compare into data/
python src/app.py --rebuild
```

Tip: start with 2-3 papers on a related topic (e.g. three papers proposing different approaches to the same problem) — that's where comparison questions get interesting.

config.py — Centralized settings

All the parameters you'll want to tune while experimenting live here. This makes it easy to run controlled comparisons, e.g. does a smaller chunk size improve retrieval precision?

```
"""
config.py
---------
Central place for all the knobs you'll want to tune while experimenting.
Keeping these in one file makes it easy to run controlled experiments,
e.g. "does CHUNK_SIZE=500 retrieve better than CHUNK_SIZE=1000?"
"""

import os
from pathlib import Path

# --- Paths -------------------------------------------------------------
PROJECT_ROOT = Path(__file__).resolve().parent.parent
DATA_DIR = PROJECT_ROOT / "data"
CHROMA_DB_DIR = PROJECT_ROOT / "chroma_db"

# --- Chunking ----------------------------------------------------------
# Smaller chunks -> more precise retrieval but less context per chunk.
# Larger chunks -> more context but retrieval gets noisier.
CHUNK_SIZE = 800          # characters per chunk
CHUNK_OVERLAP = 150       # overlap between consecutive chunks

# --- Embeddings --------------------------------------------------------
# Runs fully locally, no API key needed. Swap for "all-mpnet-base-v2" for
# higher quality at the cost of speed, or an OpenAI/Voyage embedding model
# if you want to compare local vs. hosted embeddings.
EMBEDDING_MODEL_NAME = "all-MiniLM-L6-v2"

# --- Retrieval ---------------------------------------------------------
# Number of chunks retrieved PER DOCUMENT, not in total. If you compare
# 3 papers with TOP_K_PER_DOC=4, the LLM will see up to 12 chunks.
TOP_K_PER_DOC = 4

# --- LLM (generation) --------------------------------------------------
ANTHROPIC_MODEL = "claude-sonnet-4-6"
ANTHROPIC_API_KEY = os.environ.get("ANTHROPIC_API_KEY", "")
MAX_TOKENS = 2000
```

STEP 4 of 8

ingest.py — Loading & chunking documents

Loads each PDF or text file and splits it into overlapping chunks. The critical detail: every chunk is wrapped in a small **Chunk** object carrying its source filename — this tag is what makes per-document retrieval possible downstream.

```
"""
ingest.py
---------
Loads raw documents (PDF or .txt) and splits them into overlapping chunks.

The key design decision for MULTI-document comparison (as opposed to
single-document RAG) is that every chunk is tagged with a `source`
field in its metadata. Without this tag, once chunks are embedded and
stored, you lose the ability to say "give me only the chunks that came
from Paper A" -- which is exactly what the comparison step needs.
"""

from pathlib import Path
from dataclasses import dataclass
from pypdf import PdfReader


@dataclass
class Chunk:
    text: str
    source: str       # e.g. "paper_a.pdf" -- this is what enables comparison
    chunk_id: str      # unique id: f"{source}::{index}"


def load_text(file_path: Path) -> str:
    """Load raw text from a .pdf or .txt file."""
    if file_path.suffix.lower() == ".pdf":
        reader = PdfReader(str(file_path))
        return "\n".join(page.extract_text() or "" for page in reader.pages)
    elif file_path.suffix.lower() in (".txt", ".md"):
        return file_path.read_text(encoding="utf-8", errors="ignore")
    else:
        raise ValueError(f"Unsupported file type: {file_path.suffix}")


def chunk_text(text: str, chunk_size: int, overlap: int) -> list[str]:
    """
    Simple fixed-size sliding-window chunker.

    This is intentionally the simplest possible strategy so the project
    is easy to reason about. A good "level up" exercise is to swap this
    for a semantic chunker (split on paragraph/sentence boundaries using
    embeddings similarity) and compare retrieval quality.
    """
    text = " ".join(text.split())  # collapse whitespace
    chunks = []
    start = 0
    while start < len(text):
        end = start + chunk_size
        chunks.append(text[start:end])
        start += chunk_size - overlap
    return [c for c in chunks if c.strip()]


def load_and_chunk_documents(data_dir: Path, chunk_size: int, overlap: int) -> list[Chunk]:
    """
    Walk a directory of source documents and return a flat list of
    Chunk objects, each tagged with which document it came from.
    """
    all_chunks: list[Chunk] = []
```

```
files = sorted(list(data_dir.glob("*.pdf")) + list(data_dir.glob("*.txt")))

if not files:
    raise FileNotFoundError(
        f"No .pdf or .txt files found in {data_dir}. "
        "Add the documents you want to compare there first."
    )

for file_path in files:
    raw_text = load_text(file_path)
    pieces = chunk_text(raw_text, chunk_size, overlap)
    for i, piece in enumerate(pieces):
        all_chunks.append(
            Chunk(text=piece, source=file_path.name, chunk_id=f"{file_path.name}::{i}")
        )
    print(f"  {file_path.name}: {len(pieces)} chunks")

return all_chunks
```

vector_store.py — Storage & per-document retrieval

Wraps ChromaDB. Rather than creating a separate collection per document (which gets messy as you add more files), everything lives in one collection, and metadata filtering (where={"source": ...}) is used to query each document independently.

```
"""
vector_store.py
---------------
Wraps ChromaDB so the rest of the app doesn't need to know about
embedding models or Chroma's API directly.

Design choice: we use ONE Chroma collection for all documents (not one
collection per document), and rely on metadata filtering (`where={"source": ...}`)
to retrieve per-document results. This is simpler to maintain and scales
to any number of documents without changing code.
"""

import chromadb
from chromadb.utils import embedding_functions
from pathlib import Path

from ingest import Chunk


class MultiDocVectorStore:
    def __init__(self, persist_dir: Path, embedding_model_name: str):
        self.client = chromadb.PersistentClient(path=str(persist_dir))
        self.embedding_fn = embedding_functions.SentenceTransformerEmbeddingFunction(
            model_name=embedding_model_name
        )
        self.collection = self.client.get_or_create_collection(
            name="documents",
            embedding_function=self.embedding_fn,
        )

    def add_chunks(self, chunks: list[Chunk]) -> None:
        """Embed and store chunks. Chroma handles the embedding call for us
        via the embedding_function configured on the collection."""
        if not chunks:
            return
        self.collection.add(
            ids=[c.chunk_id for c in chunks],
            documents=[c.text for c in chunks],
            metadatas=[{"source": c.source} for c in chunks],
        )

    def list_sources(self) -> list[str]:
        """Return the distinct document names currently stored."""
        result = self.collection.get(include=["metadatas"])
        return sorted({m["source"] for m in result["metadatas"]})

    def query_per_document(self, query: str, sources: list[str], top_k: int) -> dict[str,
    list[str]]:
        """
        The core of multi-document RAG: run the SAME query against EACH
        source document independently (using a metadata filter), so every
        document gets a fair chance to contribute chunks -- instead of one
        document dominating the top-k because it happens to be larger or
        more textually similar to the query.
        """
        results: dict[str, list[str]] = {}
        for source in sources:
            res = self.collection.query(
                query_texts=[query],
                n_results=top_k,
```

```
        where={"source": source},
    )
    results[source] = res["documents"][0] if res["documents"] else []
return results
```

compare.py — Building the comparison prompt

This is the part that actually makes the app a *comparison* tool rather than plain RAG. Chunks are grouped under clear per-source headers, and the model is explicitly told to structure its answer source-by-source before synthesizing.

```
"""
compare.py
----------
Builds a structured, source-attributed prompt from the per-document
retrieval results and calls the LLM to produce a comparison.

The prompt design is the single most important part of a multi-document
RAG system. Two rules make it work well:

1. Keep chunks grouped and clearly labeled by source. Never interleave
   chunks from different documents without labels -- the model needs
   to know "this paragraph is from Paper A" to compare correctly.
2. Explicitly instruct the model to cite which document each claim comes
   from. This both improves faithfulness and makes answers verifiable.
"""

import anthropic
import config


def build_comparison_prompt(question: str, retrieved: dict[str, list[str]]) -> str:
    sections = []
    for source, chunks in retrieved.items():
        if not chunks:
            continue
        joined = "\n\n".join(f"[Excerpt {i+1}] {c}" for i, c in enumerate(chunks))
        sections.append(f"=== SOURCE: {source} ===\n{joined}")

    context_block = "\n\n".join(sections)

    prompt = f"""You are comparing multiple source documents to answer a question.

Below are excerpts retrieved from each document. Excerpts from the same
document are grouped under a "SOURCE:" header.

{context_block}

QUESTION: {question}

Instructions:
- Answer using ONLY the excerpts above. If the excerpts don't contain
  enough information to fully answer, say so explicitly.
- Structure your answer by clearly stating what each source says before
  drawing comparisons, e.g. "Paper A approaches this by... Paper B, in
  contrast, ..."
- End with a short synthesis: where do the sources agree, and where do
  they genuinely differ?
- Every claim should be traceable to a specific source. Reference the
  source name when making a claim (e.g. "According to paper_a.pdf...").
"""
    return prompt


def get_comparison(question: str, retrieved: dict[str, list[str]]) -> str:
    client = anthropic.Anthropic(api_key=config.ANTHROPIC_API_KEY)
    prompt = build_comparison_prompt(question, retrieved)

    response = client.messages.create(
        model=config.ANTHROPIC_MODEL,
        max_tokens=config.MAX_TOKENS,
```

```
    messages=[{"role": "user", "content": prompt}],
)
return response.content[0].text
```

app.py — Putting it all together (CLI)

The entry point: indexes documents (if not already indexed), then loops asking the user for comparison questions and printing formatted answers.

```
"""
app.py
------
Command-line entry point that ties everything together:

    1. Ingest + chunk documents from data/
    2. Embed and store chunks in Chroma (skipped if already indexed)
    3. Loop: take a question, retrieve per-document, ask the LLM to compare
    4. Print a nicely formatted answer

Run with:
    python src/app.py --rebuild      # re-index documents from scratch
    python src/app.py                # reuse existing index
"""

import argparse
import shutil

from rich.console import Console
from rich.markdown import Markdown
from rich.panel import Panel

import config
from ingest import load_and_chunk_documents
from vector_store import MultiDocVectorStore
from compare import get_comparison

console = Console()


def build_index(rebuild: bool) -> MultiDocVectorStore:
    if rebuild and config.CHROMA_DB_DIR.exists():
        shutil.rmtree(config.CHROMA_DB_DIR)

    store = MultiDocVectorStore(config.CHROMA_DB_DIR, config.EMBEDDING_MODEL_NAME)

    if rebuild or not store.list_sources():
        console.print("[bold cyan]Indexing documents...[/bold cyan]")
        chunks = load_and_chunk_documents(
            config.DATA_DIR, config.CHUNK_SIZE, config.CHUNK_OVERLAP
        )
        store.add_chunks(chunks)
    else:
        console.print("[dim]Using existing index. Pass --rebuild to re-index.[/dim]")

    return store


def main():
    parser = argparse.ArgumentParser()
    parser.add_argument("--rebuild", action="store_true", help="Re-index documents from scratch")
    args = parser.parse_args()

    store = build_index(args.rebuild)
    sources = store.list_sources()

    if len(sources) < 2:
        console.print(
            "[bold red]Warning:[/bold red] fewer than 2 documents indexed. "
            "Add more files to data/ to make comparisons meaningful."
        )
```

```
    console.print(Panel(f"Indexed documents: {', '.join(sources)}", title="Multi-Doc RAG"))

    while True:
        question = console.input("\n[bold green]Ask a comparison question[/bold green] (or
    'quit'): ")
        if question.strip().lower() in ("quit", "exit"):
            break

        retrieved = store.query_per_document(question, sources, config.TOP_K_PER_DOC)
        answer = get_comparison(question, retrieved)

        console.print(Markdown(answer))


if __name__ == "__main__":
    main()
```

STEP 8 of 8

Example Run & Extensions

Example session

```
$ python src/app.py --rebuild
Indexing documents...
  paper_a.pdf: 42 chunks
  paper_b.pdf: 38 chunks
  paper_c.pdf: 51 chunks

Indexed documents: paper_a.pdf, paper_b.pdf, paper_c.pdf

Ask a comparison question (or 'quit'): How do these papers evaluate their models?

According to paper_a.pdf, evaluation relies primarily on accuracy and F1 score on a
held-out test set... paper_b.pdf, in contrast, emphasizes human evaluation via
crowdsourced ratings... paper_c.pdf combines both automatic metrics and a small-scale
user study...

Synthesis: all three papers use at least one automatic metric, but only paper_b.pdf
and paper_c.pdf incorporate human judgment, suggesting the field has not converged
on a single evaluation standard.
```

Ways to extend this into a stronger portfolio project

- **Citations UI:** build a small Streamlit front-end that highlights the exact retrieved passage next to each claim.
- **Hybrid search:** combine the vector search with BM25 keyword search and compare retrieval quality.
- **Evaluation harness:** use a framework like RAGAS to measure retrieval precision/recall and answer faithfulness across chunk sizes.
- **Semantic chunking:** replace the fixed-size chunker with one that splits on paragraph/topic boundaries.
- **N-way scaling:** test with 5-10 documents instead of 2-3, and check whether the prompt needs summarization before comparison to fit context limits.